



Understanding The constexpr 2-Step From Compile Time To Run Time

Jason Turner

2024

Understanding The `constexpr` Two-Step

Jason Turner

- Host of C++ Weekly <https://www.youtube.com/@cppweekly>
- Author
 - C++ Best Practices
 - OpCode, Copy and Reference, Object Lifetime Puzzlers
 - <https://amzn.to/3xWh8Ox>
 - https://leanpub.com/u/jason_turner
- Developer
 - <https://cppbestpractices.com>
 - <https://github.com/lefticus>
 - <https://github.com/cpp-best-practices>
- Microsoft MVP for C++, 2015-present

Jason Turner

Independent and available for training and code reviews

- <https://articles.emptycrate.com/idocpp>

About my Talks

- Move to the front!
- Please interrupt and ask questions
- This is approximately how my training days look

Understanding The `constexpr` Two-Step

Notes:

Understanding The `constexpr` Two-Step

Notes:

- I went to an extreme effort to try and fit meaningful and readable examples on single slides. Hopefully it worked out.

Understanding The `constexpr` Two-Step

Notes:

- I went to an extreme effort to try and fit meaningful and readable examples on single slides. Hopefully it worked out.
- There's a lot of code on the slides

Understanding The `constexpr` Two-Step

Notes:

- I went to an extreme effort to try and fit meaningful and readable examples on single slides. Hopefully it worked out.
- There's a lot of code on the slides
- If you don't ask questions, we'll probably move too fast

C++20's `constexpr` Containers

`constexpr` `new` and `delete` (C++20)

- GCC 10 (May 2020)
- Clang 10 (March 2020)
- VS 16.9 (March 2021)

`constexpr` `std::vector` and `std::string` `constexpr` (C++20)

- GCC 12 (May 2022)
- Clang 15 (Sept 2022)
- VS 16.10 (May 2021)

`std::vector` is `constexpr`?

`std::vector` is `constexpr`?

So I can do this, right?

```
1  #include <vector>
2
3  int main()
4  {
5      constexpr std::vector<int> data{1,2,3,4,5};
6  }
```

<https://godbolt.org/z/zx1vEGzcr>

C++20's `constexpr` Containers

- Who has attempted to use `std::vector` or `std::string` in `constexpr` code?

C++20's `constexpr` Containers

- Who has attempted to use `std::vector` or `std::string` in `constexpr` code?
- Who has successfully used `std::vector` or `std::string` in `constexpr` code?

`std::vector` is `constexpr` (but not how you think)

```
1 | constexpr std::vector<int> data{1,2,3,4,5};
```

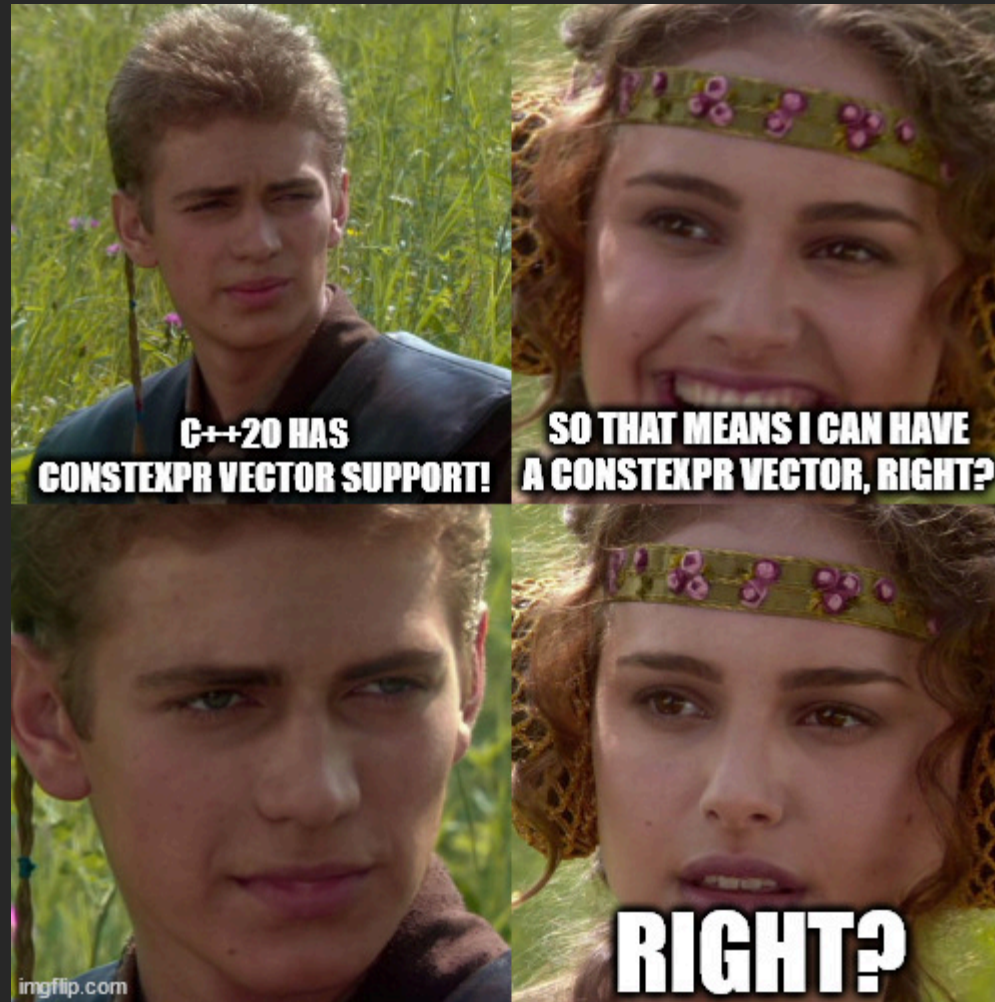
`std::vector` is `constexpr` (but not how you think)

```
1 | constexpr std::vector<int> data{1,2,3,4,5};
```

```
1 | <source>:5:32: error: constexpr variable 'data' must be initialized by a constant
2 | expression
3 |     5 |         constexpr std::vector<int> data{1,2,3,4,5};
4 |         |                                     ^~~~~~
5 | <source>:5:32: note: pointer to subobject of heap-allocated object is not a constant
6 | expression
7 | /opt/compiler-explorer/clang-trunk-
8 | 20240702/bin/./include/c++/v1/__memory/allocator.h:116:32: note: heap allocation
9 | performed here
   | 116 |         return static_cast<Tp*>(::operator new(__n * sizeof(_Tp)));
   |         |
1 | 1 error generated.
   | Compiler returned: 1
```

`std::vector` is `constexpr`?

`std::vector` is `constexpr`?



`constexpr`

`std::vector`

Does this compile?

```
1  #include <vector>
2
3  int main()
4  {
5      constexpr std::vector<int> data{};
6  }
```

<https://godbolt.org/z/qG4Mcnah5>

`constexpr`

`std::vector`

Does this compile?

```
1  #include <vector>
2
3  int main()
4  {
5      constexpr std::vector<int> data{};
6  }
```

<https://godbolt.org/z/qG4Mcnah5>

Yes.

What is Happening Here?

`constexpr` and Dynamic Lifetime Objects

[`expr.const`]

Effectively: memory allocated at compile time must be freed at compile time.

Is this a valid function?

```
1 constexpr int *make_data() {  
2     return new int(42);  
3 }
```

Is this a valid function?

```
1 constexpr int *make_data() {  
2     return new int(42);  
3 }
```

Sure, this is a function that will compile.

Is this valid?

```
1 constexpr int *make_data() {  
2     return new int(42);  
3 }  
4  
5 constexpr int use_data() {  
6     return *make_data();  
7 }
```

<https://godbolt.org/z/5TsM7jPvx>

Is this valid?

```
1 constexpr int *make_data() {  
2     return new int(42);  
3 }  
4  
5 constexpr int use_data() {  
6     return *make_data();  
7 }
```

<https://godbolt.org/z/5TsM7jPvx>

Sure, this will compile.

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     return use_data();  
11 }
```

<https://godbolt.org/z/bKPGPT6PK>

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     return use_data();  
11 }
```

<https://godbolt.org/z/bKPGPT6PK>

Sure, this will compile.

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     int i = use_data();  
11     return i;  
12 }
```

<https://godbolt.org/z/qY683MdWM>

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     int i = use_data();  
11     return i;  
12 }
```

<https://godbolt.org/z/qY683MdWM>

Sure, this will compile.

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     constexpr int i = use_data();  
11     return i;  
12 }
```

<https://godbolt.org/z/rWEbY3hc9>

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     constexpr int i = use_data();  
11     return i;  
12 }
```

<https://godbolt.org/z/rWEbY3hc9>

No, because we allocated a new `int` at compile-time and didn't free it.

Is this valid?

```
1  constexpr int *make_data() {  
2      return new int(42);  
3  }  
4  
5  constexpr int use_data() {  
6      return *make_data();  
7  }  
8  
9  int main() {  
10     constexpr int i = use_data();  
11     return i;  
12 }
```

<https://godbolt.org/z/rWEbY3hc9>

No, because we allocated a new `int` at compile-time and didn't free it.

How do we make it valid?

The “Two-Step”

```
1  constexpr int *make_data() {  
2  
3      return new int(42);  
4  }  
5  
6  constexpr int use_data() {  
7      int *data = make_data();  
8  
9      int result = *data;  
10     delete data;  
11     return result;  
12 }  
13  
14 int main() {  
15     constexpr int i = use_data();  
16     return i;  
17 }
```

<https://godbolt.org/z/3rMMGYTMs>

The “Two-Step”

```
1  constexpr int *make_data() {  
2  /// 1) dynamic storage thing at compile-time  
3  return new int(42); ///  
4  }  
5  
6  constexpr int use_data() {  
7      int *data = make_data();  
8  
9      int result = *data;  
10     delete data;  
11     return result;  
12 }  
13  
14 int main() {  
15     constexpr int i = use_data();  
16     return i;  
17 }
```

<https://godbolt.org/z/rYMrzdrse>

The “Two-Step”

```
1  constexpr int *make_data() {  
2      // 1) dynamic storage thing at compile-time  
3      return new int(42);  
4  }  
5  
6  constexpr int use_data() {  
7      int *data = make_data();  
8      /// 2) runtime usable copy made  
9      int result = *data; ///  
10     delete data;  
11     return result;  
12 }  
13  
14 int main() {  
15     constexpr int i = use_data();  
16     return i;  
17 }
```

<https://godbolt.org/z/zW7cPqhT9>

The “Two-Step”

The “Two-Step”

1. Do all the dynamic storage stuff you want to at compile-time

The “Two-Step”

1. Do all the dynamic storage stuff you want to at compile-time
2. Copy the dynamic storage stuff to static stuff at compile-time (and make sure you free the dynamic thing at compile time)

Thank You! Questions?

Goals

Goals

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>`

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>`
- Level 2: `std::string` -> `constexpr std::string_view`

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>`
- Level 2: `std::string` -> `constexpr std::string_view`
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array)

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>`
- Level 2: `std::string` -> `constexpr std::string_view`
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array)
- Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Level 1 `vector` -> `oversized_array`

```
1  #include <array>
2
3  template <typename ValueType, std::size_t Capacity = 255>
4  struct oversized_array {
5      std::array<ValueType, Capacity> data{}; // default init on purpose
6      std::size_t size;
7      [[nodiscard]] constexpr auto begin() const noexcept { return data.begin(); }
8      [[nodiscard]] constexpr auto end() const noexcept {
9          return std::next(data.begin(), static_cast<std::ptrdiff_t>(size));
10     }
11 };
```

<https://godbolt.org/z/Ye6M6zhhb>

- Note: `begin` and `end` are all that's necessary to make this ranged-for capable.

Level 1 `vector` -> `oversized_array`

```
1  #include <array>
2
3  // compacted version
4  template <typename ValueType, std::size_t Capacity = 255>
5  struct oversized_array {
6      std::array<ValueType, Capacity> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
```

<https://godbolt.org/z/vf5dffsz5>

- removed `[[nodiscard]]` and `noexcept` and `std::next` to make code smaller

The Function We Want To Use

```
1 #include <vector>
2
3 constexpr std::vector<double> make_angles(int count) {
4     std::vector<double> result;
5     for (int i = 0; i < count; ++i) { result.push_back(360.0/count * i); }
6     return result;
7 }
```

<https://godbolt.org/z/aneahbnq4>

The Function We Want To Use

```
1  #include <vector>
2
3  constexpr std::vector<double> make_angles(int count) {
4      std::vector<double> result;
5      for (int i = 0; i < count; ++i) { result.push_back(360.0/count * i); }
6      return result;
7  }
```

<https://godbolt.org/z/aneahbnq4>

- Does this look like a reasonable thing?

Why Not Do This?

```
1  #include <array>
2
3  template<std::size_t Count>
4  constexpr std::array<double, Count> make_angles() {
5      std::array<double, Count> result;
6      for (int i = 0; i < Count; ++i) { result[i] = 360.0/Count * i; }
7      return result;
8  }
```

<https://godbolt.org/z/jM8GTWEvK>

Pros / Cons of doing this?

Why Not Do This?

```
1  #include <array>
2
3  template<std::size_t Count>
4  constexpr std::array<double, Count> make_angles() {
5      std::array<double, Count> result;
6      for (int i = 0; i < Count; ++i) { result[i] = 360.0/Count * i; }
7      return result;
8  }
```

<https://godbolt.org/z/jM8GTWEvK>

Pros / Cons of doing this?

- Size is perfectly known at compile time

Why Not Do This?

```
1  #include <array>
2
3  template<std::size_t Count>
4  constexpr std::array<double, Count> make_angles() {
5      std::array<double, Count> result;
6      for (int i = 0; i < Count; ++i) { result[i] = 360.0/Count * i; }
7      return result;
8  }
```

<https://godbolt.org/z/jM8GTWEvK>

Pros / Cons of doing this?

- Size is perfectly known at compile time
- But size must be known at compile time

Why Not Do This?

```
1  #include <array>
2
3  template<std::size_t Count>
4  constexpr std::array<double, Count> make_angles() {
5      std::array<double, Count> result;
6      for (int i = 0; i < Count; ++i) { result[i] = 360.0/Count * i; }
7      return result;
8  }
```

<https://godbolt.org/z/jM8GTWEvK>

Pros / Cons of doing this?

- Size is perfectly known at compile time
- But size must be known at compile time
- (This function isn't really usable at runtime)

Why Not Do This?

```
1  #include <array>
2
3  template<std::size_t Count>
4  constexpr std::array<double, Count> make_angles() {
5      std::array<double, Count> result;
6      for (int i = 0; i < Count; ++i) { result[i] = 360.0/Count * i; }
7      return result;
8  }
```

<https://godbolt.org/z/jM8GTWEvK>

Pros / Cons of doing this?

- Size is perfectly known at compile time
- But size must be known at compile time
- (This function isn't really usable at runtime)

Aside: I like to think of `constexpr` functions as regular functions that can be also executed at compile time, not as “meta programming”

Level 1 Attempt 1

```

1  #include <array>
2  #include <iostream>
3  #include <vector>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP = 255> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr std::vector<double> make_angles(int count) {
12      std::vector<double> result;
13      for (int i = 0; i < count; ++i) { result.push_back(360.0/count * i); }
14      return result;
15  }
16
17  template<typename VT> constexpr auto to_oversized(const std::vector<VT> &input) {
18      oversized_array<VT> result{}, input.size();
19      std::ranges::copy(input, result.data.begin());
20      return result;
21  }
22
23  int main() {
24      constexpr static auto data = to_oversized(make_angles(300));
25      for (const auto &val : data) { std::cout << val; }
26  }
27  /// Look Good? (subtle bug, who sees it?)

```

<https://godbolt.org/z/caGavso4z>

Level 1 Attempt 2

```

1  #include <array>
2  #include <iostream>
3  #include <vector>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr std::vector<double> make_angles(int count) {
12      std::vector<double> result;
13      for (int i = 0; i < count; ++i) { result.push_back(360.0/count * i); }
14      return result;
15  }
16
17  template<std::size_t CAP, typename VT> ///
18  constexpr auto to_oversized(const std::vector<VT> &input) {
19      oversized_array<VT, CAP> result{{}, input.size()};
20      std::ranges::copy(input, result.data.begin());
21      return result;
22  }
23
24  int main() {
25      constexpr static auto data = to_oversized<512>(make_angles(300)); ///
26      for (const auto &val : data) { std::cout << val; }
27  }
28  /// Level 1 Complete?

```

<https://godbolt.org/z/sx4x7Gd61>

**Level 1: Successfully copied
from a dynamic storage
thing to a fixed thing.**

Level 1

- Yes this is simple and we can clearly see the needed sizes, but the point is to think more generically as a building block
- The binary size can grow unfortunately large if we aren't careful

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>` ✓
- Level 2: `std::string` -> `constexpr std::string_view`
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array)
- Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Level 2: `std::string` ->

`constexpr std::string_view`

Level 2: `std::string` -> `constexpr`
`std::string_view`

Level 2: `std::string` -> `constexpr` `std::string_view`

- What is `std::string_view`?

Level 2: `std::string` -> `constexpr` `std::string_view`

- What is `std::string_view`?
 - Pointer + Size

Level 2: `std::string` -> `constexpr`

`std::string_view`

- What is `std::string_view`?
 - Pointer + Size
- How do we ensure `std::string_view` is used safely?

Level 2: `std::string` -> `constexpr`

`std::string_view`

- What is `std::string_view`?
 - Pointer + Size
- How do we ensure `std::string_view` is used safely?
 - The data must outlive the `string_view`

Level 2: `std::string` -> `constexpr` `std::string_view`

- What is `std::string_view`?
 - Pointer + Size
- How do we ensure `std::string_view` is used safely?
 - The data must outlive the `string_view`
- So we need a data store, do we have one that will work?

Level 2: `std::string` -> `constexpr` `std::string_view`

- What is `std::string_view`?
 - Pointer + Size
- How do we ensure `std::string_view` is used safely?
 - The data must outlive the `string_view`
- So we need a data store, do we have one that will work?
 - Yes, our `oversized_array<char>` can work.

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     oversized_array<char, 255> oss{}, input.size();
14     std::ranges::copy(input, oss.data.begin());
15     return std::string_view{oss.begin(), oss.end()};
16 }
17 int main() {
18     auto string_like = make_sv(make_string(42));
19     std::cout << string_like;
20 }

```

<https://godbolt.org/z/cbzeq18sd>

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     oversized_array<char, 255> oss{{}}, input.size();
14     std::ranges::copy(input, oss.data.begin());
15     return std::string_view{oss.begin(), oss.end()};
16 }
17 int main() {
18     auto string_like = make_sv(make_string(42));
19     std::cout << string_like;
20 }

```

<https://godbolt.org/z/cbzeq18sd>

- What is printed and why? (subtle again)

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     oversized_array<char, 255> oss{{}}, input.size();
14     std::ranges::copy(input, oss.data.begin());
15     return std::string_view{oss.begin(), oss.end()};
16 }
17 int main() {
18     auto string_like = make_sv(make_string(42));
19     std::cout << string_like;
20 }

```

<https://godbolt.org/z/cbzeq18sd>

- What is printed and why? (subtle again)
- We are returning reference to local data.

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     constexpr auto oss = [&]{
14         oversized_array<char, 255> result{}, input.size();
15         std::ranges::copy(input, result.data.begin());
16         return result;
17     }();
18     return std::string_view{oss.begin(), oss.end()};
19 }
20 int main() {
21     auto string_like = make_sv(make_string(42));
22     std::cout << string_like;
23 }

```

<https://godbolt.org/z/6c6Mh3KvE>

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     constexpr auto oss = [&]{
14         oversized_array<char, 255> result{{}, input.size()};
15         std::ranges::copy(input, result.data.begin());
16         return result;
17     }();
18     return std::string_view{oss.begin(), oss.end()};
19 }
20 int main() {
21     auto string_like = make_sv(make_string(42));
22     std::cout << string_like;
23 }

```

<https://godbolt.org/z/6c6Mh3KvE>

- What about now?

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv(const std::string &input) {
13     constexpr auto oss = [&]{
14         oversized_array<char, 255> result{{}, input.size()};
15         std::ranges::copy(input, result.data.begin());
16         return result;
17     }();
18     return std::string_view{oss.begin(), oss.end()};
19 }
20 int main() {
21     auto string_like = make_sv(make_string(42));
22     std::cout << string_like;
23 }

```

<https://godbolt.org/z/6c6Mh3KvE>

- What about now?
- Cannot compile because (as of C++23 at least) parameters are never considered to be `constexpr`.

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv() {
13     constexpr auto oss = [&]{
14         const auto input = make_string(42); /// fix it?
15         oversized_array<char, 255> result{{}, input.size()};
16         std::ranges::copy(input, result.data.begin());
17         return result;
18     }();
19     return std::string_view{oss.begin(), oss.end()};
20 }
21 int main() {
22     auto string_like = make_sv();
23     std::cout << string_like;
24 }

```

<https://godbolt.org/z/jYroP6cqo>

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv() {
13     constexpr auto oss = [&]{
14         const auto input = make_string(42); /// fix it?
15         oversized_array<char, 255> result{{}, input.size()};
16         std::ranges::copy(input, result.data.begin());
17         return result;
18     }();
19     return std::string_view{oss.begin(), oss.end()};
20 }
21 int main() {
22     auto string_like = make_sv();
23     std::cout << string_like;
24 }

```

<https://godbolt.org/z/jYroP6cqo>

`constexpr` does not affect object lifetime, the `string_view` returned is pointing to an object whose lifetime has ended.

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv() {
13     constexpr auto oss = [&]{
14         const auto input = make_string(42); ///
15         oversized_array<char, 255> result{{}, input.size()};
16         std::ranges::copy(input, result.data.begin());
17         return result;
18     }();
19     return std::string_view{oss.begin(), oss.end()};
20 }
21 int main() {
22     constexpr auto string_like = make_sv(); /// fails to compile now
23     std::cout << string_like;
24 }

```

<https://godbolt.org/z/Gra9465aT>

Pro Tip: When Making Your
Data->View Converters,
Make Them `constexpr`

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 constexpr std::string_view make_sv() { ///
13     constexpr auto oss = [&]{
14         const auto input = make_string(42);
15         oversized_array<char, 255> result{{}, input.size()};
16         std::ranges::copy(input, result.data.begin());
17         return result;
18     }();
19     return std::string_view{oss.begin(), oss.end()};
20 }
21 int main() {
22     auto string_like = make_sv(); /// fails to compile now
23     std::cout << string_like;
24 }

```

<https://godbolt.org/z/3Ee8nndar>

```

1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10 constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
11
12 consteval std::string_view make_sv() {
13     constexpr static auto oss = [&]{ /// C++23
14         const auto input = make_string(42);
15         oversized_array<char, 255> result{{}, input.size()};
16         std::ranges::copy(input.begin(), input.end(), result.data.begin());
17         return result;
18     }();
19     return std::string_view{oss.begin(), oss.end()};
20 }
21 int main() {
22     auto string_like = make_sv(); /// compiles!
23     std::cout << string_like;
24 }

```

<https://godbolt.org/z/hEa8a8Ycx>

Level 2 (local **static**)

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  int main() {
14      constexpr static auto oss = [&]{
15          const auto input = make_string(42);
16          oversized_array<char, 255> result{{}, input.size()};
17          std::ranges::copy(input, result.data.begin());
18          return result;
19      }();
20      constexpr static std::string_view string_like{oss.begin(), oss.end()};
21
22      std::cout << string_like;
23  }
```

<https://godbolt.org/z/9oaWTsaK7>

Level 2

```
1  constexpr std::string_view make_sv() {  
2      constexpr static auto oss = [&]{  
3          const auto input = make_string(42); /// this is annoying  
4          oversized_array<char, 255> result{{}, input.size()};  
5          std::ranges::copy(input, result.data.begin());  
6          return result;  
7      }();  
8      return std::string_view{oss.begin(), oss.end()};  
9  }
```

<https://godbolt.org/z/eYednzoe5>

How do we fix this hard coded issue?

Level 2 (builder)

Template parameter

```
1  template<auto Builder> consteval auto to_view() {
2      constexpr static auto oss = []{
3          const auto input = Builder(); ///
4          oversized_array<char, 255> result{{}, input.size()};
5          std::ranges::copy(input, result.data.begin());
6          return result;
7      }();
8      return std::string_view{oss.begin(), oss.end()};
9  }
```

<https://godbolt.org/z/WqMY4xEh5>

Function parameter

```
1  consteval auto to_view(auto Builder) {
2      constexpr static auto oss = []{
3          const auto input = Builder(); ///
4          oversized_array<char, 255> result{{}, input.size()};
5          std::ranges::copy(input, result.data.begin());
6          return result;
7      }();
8      return std::string_view{oss.begin(), oss.end()};
9  }
```

<https://godbolt.org/z/P4zxYvsqv>

Level 2 (builder)

```
1 | to_view<[x=42]() { return make_string(x); }>();
```

```
1 | to_view([x=42]() { return make_string(x); });
```

We've added a capture - which of these compiles (if any)?

Level 2 (builder)

```
1 | to_view<[x=42]() { return make_string(x); }>();
```

```
1 | to_view([x=42]() { return make_string(x); });
```

We've added a capture - which of these compiles (if any)?

Only the first one does - the lambda is no longer callable (basically because it no longer decays to a function pointer and is therefore not a constant expression)

Level 2 (builder)

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder(); ///
16          oversized_array<char, 255> result{{}, input.size()};
17          std::ranges::copy(input, result.data.begin());
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()};
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>();
25      std::cout << string_like;
26  }
```

Level 2 complete?

Level 2 Catchup / Questions

Level 2 Catchup

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder();
16          oversized_array<char, 255> result{{}, input.size()};
17          std::ranges::copy(input, result.data.begin());
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()};
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>(); /// 1
25      std::cout << string_like;
26  }
```

<https://godbolt.org/z/Gq9n84Kc3>

Level 2 Catchup

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder(); /// 2
16          oversized_array<char, 255> result{{}, input.size()};
17          std::ranges::copy(input, result.data.begin());
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()};
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>(); // 1
25      std::cout << string_like;
26  }
```

<https://godbolt.org/z/E4dnnovr>

Level 2 Catchup

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder(); // 2
16          oversized_array<char, 255> result{{}, input.size()}; /// 3 (size)
17          std::ranges::copy(input, result.data.begin());
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()};
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>(); // 1
25      std::cout << string_like;
26  }
```

<https://godbolt.org/z/M5rEMs9z1>

Level 2 Catchup

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder(); // 2
16          oversized_array<char, 255> result{{}, input.size()}; // 3 (size)
17          std::ranges::copy(input, result.data.begin()); /// 4 (copy)
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()};
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>(); // 1
25      std::cout << string_like;
26  }
```

<https://godbolt.org/z/ddc1jjcPG>

Level 2 Catchup

```
1  #include <iostream>
2  #include <array>
3  #include <string>
4  #include <algorithm>
5  template <typename VT, std::size_t CAP> struct oversized_array {
6      std::array<VT, CAP> data{};  std::size_t size;
7      constexpr auto begin() const { return data.begin(); }
8      constexpr auto end() const { return data.begin() + size; }
9  };
10
11  constexpr auto make_string(std::size_t num) { return std::string(num, 'a'); }
12
13  template<auto Builder> consteval auto to_view() {
14      constexpr static auto oss = []{
15          const auto input = Builder(); // 2
16          oversized_array<char, 255> result{{}, input.size()}; // 3 (size)
17          std::ranges::copy(input, result.data.begin()); // 4 (copy)
18          return result;
19      }();
20      return std::string_view{oss.begin(), oss.end()}; /// 5 (view to static data)
21  }
22
23  int main() {
24      static constexpr auto string_like = to_view<[] { return make_string(42); }>(); // 1
25      std::cout << string_like;
26  }
```

<https://godbolt.org/z/4nrrTrGo8>

What's Kind of Like a `string_view` But Isn't and It's From C++20?

std::span

Level 2.5 - `span` and `string_view`

An Updated “oversized”

```
1  template <typename VT, std::size_t CAP> struct oversized_array {
2      std::array<VT, CAP> data{};  std::size_t size;
3      constexpr auto begin() const { return data.begin(); }
4      constexpr auto end() const { return data.begin() + size; }
5  };
```

<https://godbolt.org/z/cY6ra4fzh>

```
1  template <typename VT, std::size_t CAP=255> struct os {
2      std::array<VT, CAP> data{};  std::size_t size; bool string;
3
4      constexpr os(const std::ranges::range auto &input, bool string)
5          : size(std::ranges::size(input)), string(string) {
6          std::ranges::copy(input, data.begin());  }
7
8  /// delegate to above constructor
9  constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
10 constexpr os(const std::vector<VT> &in) : os(in, false) {}
11
12 constexpr auto begin() const { return data.begin(); }
13 constexpr auto end() const { return data.begin() + size; }
14 };
```

<https://godbolt.org/z/Yf8rnqhfK>

```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17
18  template<auto Builder> consteval auto to_view() {
19      static constexpr auto oss = os(Builder());
20      if constexpr (oss.string) {
21          return std::string_view(oss);
22      } else {
23          return std::span(oss);
24      }
25  }
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>();
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```


Level 2.5 - `span` and `string_view`

Note: I found a bug in clang while doing this and learned that `constexpr` `static` inside of a `constexpr` function results in the object being removed.

Mateusz Pusz found it back in March 2024: <https://github.com/llvm/llvm-project/issues/82994>

```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size;  bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17
18  template<auto Builder> consteval auto to_view() {
19      static constexpr auto oss = os(Builder()); ///
20      if constexpr (oss.string) {
21          return std::string_view(oss);
22      } else {
23          return std::span(oss);
24      }
25  }
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>();
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() { ///
18      if constexpr (Value.string) {
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); }
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>();
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```


Level 2.5 Catch Up

```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() {
18      if constexpr (Value.string) {
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); }
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); /// 1
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size;  bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() {
18      if constexpr (Value.string) {
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); } /// 2
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); // 1
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12     constexpr os(const std::basic_string<VT> &in) : os(in, true) {} /// 3
13     constexpr os(const std::vector<VT> &in) : os(in, false) {}
14     constexpr auto begin() const { return data.begin(); }
15     constexpr auto end() const { return data.begin() + size; }
16 };
17 template<os Value> consteval auto view_helper() {
18     if constexpr (Value.string) {
19         return std::string_view(Value);
20     } else {
21         return std::span(Value);
22     }
23 }
24 template<auto Builder>
25 consteval auto to_view() { return view_helper<os{Builder()}>(); } // 2
26
27 int main() {
28     static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); // 1
29     static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30 }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {} // 3
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() { /// 4
18      if constexpr (Value.string) {
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); } // 2
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); // 1
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size;  bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {} // 3
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() { // 4
18      if constexpr (Value.string) { /// 5
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); } // 2
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); // 1
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```



```

1  #include <array>
2  #include <string>
3  #include <vector>
4  #include <span>
5  #include <algorithm>
6
7  template <typename VT, std::size_t CAP=255> struct os {
8      std::array<VT, CAP> data{};  std::size_t size;  bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {} // 3
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() { // 4
18      if constexpr (Value.string) { // 5
19          return std::string_view(Value); /// 6 (then repeat for vector)
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() { return view_helper<os{Builder()}>(); } // 2
26
27  int main() {
28      static constexpr auto sv = to_view<[] { return std::string(42, 'a'); }>(); // 1
29      static constexpr auto span = to_view<[] { return std::vector<int>(42); }>();
30  }

```


Level 2 - Questions?

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>` ✓
- Level 2: `std::string` -> `constexpr std::string_view` ✓
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array)
- Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Level 3 - Right-Size The Array

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Thoughts?

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Thoughts?

1. Perform work and get vector/string back

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Thoughts?

1. Perform work and get vector/string back
2. Copy vector/string into automatic lifetime (stack allocated) over-sized thing

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Thoughts?

1. Perform work and get vector/string back
2. Copy vector/string into automatic lifetime (stack allocated) over-sized thing
3. Copy over-sized thing into static right-sized thing

Level 3 - Right-Size The Array

How do we get from over-sized thing to right-sized thing?

Thoughts?

1. Perform work and get vector/string back
2. Copy vector/string into automatic lifetime (stack allocated) over-sized thing
3. Copy over-sized thing into static right-sized thing
4. Return view to new right-sized thing

Level 3 - Right-Size The Array - The Key Bits

```
1  template<os Value> constexpr auto view_helper() {
2      // Only "Value" persists into the binary
3      if constexpr (Value.string) {
4          return std::string_view(Value);
5      } else {
6          return std::span(Value);
7      }
8  }
9  template<auto Builder>
10 constexpr auto to_view() {
11     constexpr os oversized{Builder()}; /// 1
12     // This works because oversized itself is constexpr
13     constexpr os<typename decltype(oversized)::value_type, oversized.size>
14         right_sized{oversized, oversized.string}; /// 2
15
16     return view_helper<right_sized>(); /// 3
17 }
```

<https://godbolt.org/z/q6aEWz9ee>

```

1  #include <string>
2  #include <vector>
3  #include <span>
4  #include <algorithm>
5
6  template <typename VT, std::size_t CAP=255> struct os {
7      using value_type = VT;
8      std::array<VT, CAP> data{};  std::size_t size; bool string;
9      constexpr os(const std::ranges::range auto &input, bool string)
10         : size(std::ranges::size(input)), string(string) {
11         std::ranges::copy(input, data.begin());  }
12      constexpr os(const std::basic_string<VT> &in) : os(in, true) {}
13      constexpr os(const std::vector<VT> &in) : os(in, false) {}
14      constexpr auto begin() const { return data.begin(); }
15      constexpr auto end() const { return data.begin() + size; }
16  };
17  template<os Value> consteval auto view_helper() {
18      if constexpr (Value.string) {
19          return std::string_view(Value);
20      } else {
21          return std::span(Value);
22      }
23  }
24  template<auto Builder>
25  consteval auto to_view() {
26      constexpr os oversized{Builder()};
27      constexpr os<typename decltype(oversized)::value_type, oversized.size>
28          right_sized{oversized, oversized.string};
29      return view_helper<right_sized>();
30  }

```


Level 3 - All in one flow

```

1  #include <iostream>
2  #include <vector>
3  #include <span>
4  #include <array>
5
6  template<auto Func, std::size_t Capacity> constexpr auto to_span() {
7      constexpr auto over_sized = []{ /// build oversized
8          const auto result = Func();
9          using Type = typename decltype(result)::value_type;
10         std::array<Type, Capacity> data{};
11         std::copy(std::ranges::begin(result), std::ranges::end(result), data.begin());
12         return std::pair{data, std::ranges::size(result)};
13     }();
14     static constexpr auto right_sized = [&]{ /// copy to right-sized (C++23)
15         using Type = typename decltype(over_sized.first)::value_type;
16         std::array<Type, over_sized.second> data;
17         const auto begin = over_sized.first.begin();
18         const auto end = std::next(over_sized.first.begin(), over_sized.second);
19         std::copy(begin, end, data.begin());
20         return data;
21     }();
22     return std::span(right_sized); /// return view
23 }
24 int main() {
25     constexpr auto span = to_span<[]() {
26         std::vector result{1,2,3,4}; result.push_back(42);
27         return result;
28     }, 255>();
29     for (const auto &v : span) { std::cout << v << '\n'; }
30 }

```


Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>` ✓
- Level 2: `std::string` -> `constexpr std::string_view` ✓
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array) ✓
- Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Level 4:

`std::vector<std::string>` ->

`constexpr`

`std::span<std::string_view>`

Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Taking this example input:

```
1 | std::vector<std::string> data{"Jason", "Was", "Here"};
```

What should the underlying storage for this thing look like?

Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Taking this example input:

```
1 | std::vector<std::string> data{"Jason", "Was", "Here"};
```

What should the underlying storage for this thing look like?

- `std::array<std::array<char, 5>, 3>`? (max size of contained string)

Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Taking this example input:

```
1 | std::vector<std::string> data{"Jason", "Was", "Here"};
```

What should the underlying storage for this thing look like?

- `std::array<std::array<char, 5>, 3>`? (max size of contained string)
- `std::tuple<std::array<char, 5>, std::array<char, 3>, std::array<char, 4>>`?

Level 4: `std::vector<std::string>` -> `constexpr` `std::span<std::string_view>`

Taking this example input:

```
1 | std::vector<std::string> data{"Jason", "Was", "Here"};
```

What should the underlying storage for this thing look like?

- `std::array<std::array<char, 5>, 3>`? (max size of contained string)
- `std::tuple<std::array<char, 5>, std::array<char, 3>, std::array<char, 4>>`?
- `std::array<char, 12>`?

Level 4: `std::vector<std::string>` -> `constexpr` `std::span<std::string_view>`

Taking this example input:

```
1 | std::vector<std::string> data{"Jason", "Was", "Here"};
```

What should the underlying storage for this thing look like?

- `std::array<std::array<char, 5>, 3>`? (max size of contained string)
- `std::tuple<std::array<char, 5>, std::array<char, 3>, std::array<char, 4>>`?
- `std::array<char, 12>`?
- `std::array<char, 15>`?

```

1  template<auto Builder, std::size_t Capacity> constexpr auto to_view() {
2  constexpr auto data = [&]{ ///
3      const auto input = Builder();
4      std::array<char, Capacity * Capacity> allchars{};
5      std::array<std::size_t, Capacity> string_lengths{};
6      auto current = allchars.begin();
7      auto total_chars = 0uz;
8      for (std::size_t index = 0; const auto &str : input) {
9          current = std::ranges::copy(str, current).out;
10         total_chars += string_lengths[index++] = str.size();
11     }
12     return std::tuple{input.size(), total_chars, allchars, string_lengths};
13 }();
14
15 static constexpr auto right_sized_chars = [&]{ ///
16     std::array<char, std::get<1>(data)>> result;
17     std::copy_n(std::get<2>(data).begin(), result.size(), result.begin());
18     return result;
19 }();
20
21 std::array<std::string_view, std::get<0>(data)>> views; ///
22 std::size_t start = 0;
23 for (std::size_t index = 0; index < views.size(); ++index) {
24     const auto size = std::get<3>(data)[index];
25     views[index] = std::string_view(right_sized_chars.begin() + start,
26                                     right_sized_chars.begin() + start + size);
27     start += size;
28 }
29 return views;
30 }

```


Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>`

Do we have time?

Full Example

<https://compiler-explorer.com/z/E7n1T357T>

Goals

- Level 1: `std::vector<Type>` -> `constexpr oversized_array<Type>` ✓
- Level 2: `std::string` -> `constexpr std::string_view` ✓
- Level 3: `std::vector<Type>` -> `constexpr std::array<Type, Size>` (right-sized array) ✓
- Level 4: `std::vector<std::string>` -> `constexpr`
`std::span<std::string_view>` ✓

Understanding The `constexpr` Two-Step

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

1. Work in only fixed sized stack variables (automatic objects)

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

1. Work in only fixed sized stack variables (automatic objects)
2. Work with dynamically allocated variables -> copy them to fixed sized things (point of this talk)

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

1. Work in only fixed sized stack variables (automatic objects)
2. Work with dynamically allocated variables -> copy them to fixed sized things (point of this talk)
3. Right-size the fixed sized things (level 3, optimized binary size)

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

1. Work in only fixed sized stack variables (automatic objects)
2. Work with dynamically allocated variables -> copy them to fixed sized things (point of this talk)
3. Right-size the fixed sized things (level 3, optimized binary size)
4. Create views into right-sized static objects to simplify your use case (level 4, optimized compile-time in many cases)

Understanding The `constexpr` Two-Step

Possibilities for working at compile-time.

1. Work in only fixed sized stack variables (automatic objects)
2. Work with dynamically allocated variables -> copy them to fixed sized things (point of this talk)
3. Right-size the fixed sized things (level 3, optimized binary size)
4. Create views into right-sized static objects to simplify your use case (level 4, optimized compile-time in many cases)
5. Make these things composable (level 5)

Understanding The `constexpr` Two-Step

Take aways:

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples
- Prefer `constexpr` to catch bugs earlier in the build process

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples
- Prefer `constexpr` to catch bugs earlier in the build process
- Be aware of `constexpr`/`static` bugs with clang

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples
- Prefer `constexpr` to catch bugs earlier in the build process
- Be aware of `constexpr/static` bugs with clang
- `static` inside of a `constexpr` function is C++23

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples
- Prefer `constexpr` to catch bugs earlier in the build process
- Be aware of `constexpr/static` bugs with clang
- `static` inside of a `constexpr` function is C++23
- Easy workaround for both is to make a “to_static” function with CTAD

```
template<auto Value> constexpr auto &to_static() { return Value; }
```

Understanding The `constexpr` Two-Step

Take aways:

- There are many possible approaches to these steps - I tried to vary it a bit in different examples
- Prefer `constexpr` to catch bugs earlier in the build process
- Be aware of `constexpr/static` bugs with clang
- `static` inside of a `constexpr` function is C++23
- Easy workaround for both is to make a “to_static” function with CTAD

```
template<auto Value> constexpr auto &to_static() { return Value; }
```
- Please don't try as hard as I did to make them fit on one slide, aim for more readable code!

Understanding The `constexpr` Two-Step

For more inspiration:

- https://github.com/lefticus/tools/static_views.hpp
- https://github.com/lefticus/tools/blob/main/test/static_views_tests.cpp

Note: These utilities need a view/span optimization pass like what we need in previous slides.

Understanding The `constexpr` Two-Step

```
1  const auto make_data = []() {
2      lefticus::tools::flat_map<std::string,
3                              lefticus::tools::flat_map<std::string,
4                                  std::vector<int>>> data;
5      data["hello"]["world"].push_back(42);
6      data["hello"]["jason"].push_back(72);
7      data["test"]["data"].push_back(84);
8      return data;
9  };
10
11  constexpr auto minimized = lefticus::tools::minimized_stackify<32>(make_data);
12
13  static_assert(minimized.max_size() == 2);
14  static_assert(minimized.at("hello").max_size() == 2);
15  static_assert(minimized.at("hello").at("world").capacity() == 1);
16  //
```

<https://godbolt.org/z/eoTd5fzoY>

Bugs Found During Preparation for This Talk

- GCC 10+ bug <https://compiler-explorer.com/z/oErfMPq74>
- GCC trunk bug <https://compiler-explorer.com/z/vWMnPK47Y>
- Clang bug: <https://github.com/llvm/llvm-project/issues/82994>

Understanding The `constexpr` Two-Step

Jason Turner

- Host of C++ Weekly <https://www.youtube.com/@cppweekly>
- Author
 - C++ Best Practices
 - OpCode, Copy and Reference, Object Lifetime Puzzlers
 - <https://amzn.to/3xWh8Ox>
 - https://leanpub.com/u/jason_turner
- Developer
 - <https://cppbestpractices.com>
 - <https://github.com/lefticus>
 - <https://github.com/cpp-best-practices>
- Microsoft MVP for C++, 2015-present

Jason Turner

Independent and available for training and code reviews

- <https://articles.emptycrate.com/idocpp>

